

Recurrent Neural Networks

Modeling Sequential Data

Firuz Kamalov

MAI 604: Deep Learning

May 1, 2026

Lecture Outline

- 1 Motivation & Advantages
- 2 Use Cases & Architectures
- 3 The Core Mechanics
- 4 The Loss Function
- 5 Long Short-Term Memory (LSTM)

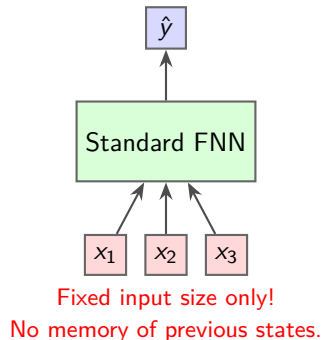
Motivation: Building a Language Model

Goal: Predict the next word in a sequence (like GPT or Gemini).

The problem with Feedforward NNs (FNN):

- FNNs require a **fixed-size input**.
- Sentences have **variable lengths**:
 - "The cat sat on the ____" (5 words)
 - "I think therefore I ____" (4 words)
- If we pad inputs, FNNs still fail to share rules across different positions.

Solution: Recurrent Neural Networks (RNN).



Advantages of RNNs

- **Variable Input Length:** RNNs process sequences step-by-step, allowing inputs of any length.
- **Parameter Sharing:** Like CNNs share filters across spatial dimensions, RNNs **share the same weights** across all time steps.
- **Memory:** They maintain an internal "hidden state" that captures information about what has been processed so far.

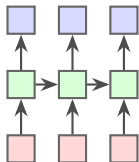
CNN vs RNN Analogy

CNNs: Apply the same W (filter) across different *pixels* to find spatial features.

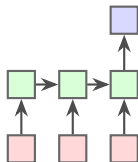
RNNs: Apply the same W (weights) across different *time steps* to find sequential features.

RNN Architectures & Real-Life Use Cases

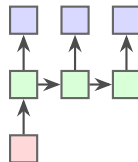
Many-to-Many
(e.g., Next-Word)



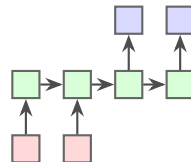
Many-to-One
(e.g., Sentiment)



One-to-Many
(e.g., Image Caption)

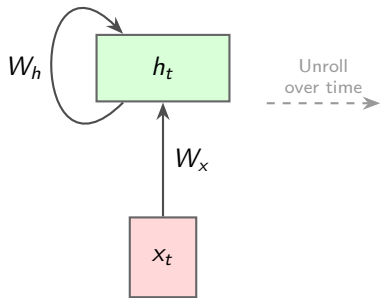


Many-to-Many
(e.g., Translation)



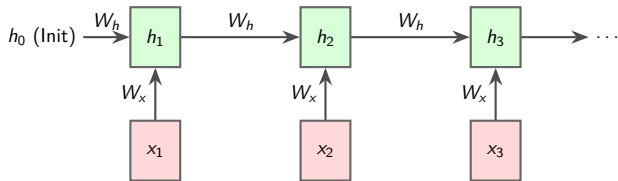
Visualizing the RNN: Hidden States

Folded (Compact)



Unroll
over time
----->

Unrolled across time



The exact same weights (W_x , W_h) are used repeatedly.

RNN Formula 1: Updating the Hidden State

At each time step t , the RNN updates its hidden memory state h_t using the current input x_t and the previous state h_{t-1} .

Hidden State Equation

$$h_t = \sigma(\mathbf{W}_h h_{t-1} + \mathbf{W}_x x_t + \mathbf{b}_h)$$

- $h_t \in \mathbb{R}^{D_h}$: Current hidden state (memory).
- $x_t \in \mathbb{R}^{D_x}$: Current input vector.
- σ : Activation function, typically tanh (squashes values to $[-1, 1]$).
- $\mathbf{W}_h$, $\mathbf{W}_x$, $\mathbf{b}_h$: Learnable weight matrices and bias.
- **Crucial**: The matrices $\mathbf{W}_h$ and $\mathbf{W}_x$ are **fixed** and reused at every time step!

Numeric Example: Computing Hidden States (1D)

Let's process a sequence of 2 numbers: $\mathbf{x} = [2.0, 1.0]$.

Assume 1D variables for simplicity:

- Fixed Weights: $W_h = -1.0$, $W_x = 0.5$, $b_h = 0.1$
- Activation $\sigma = \tanh$
- Initial state: $h_0 = 0.0$

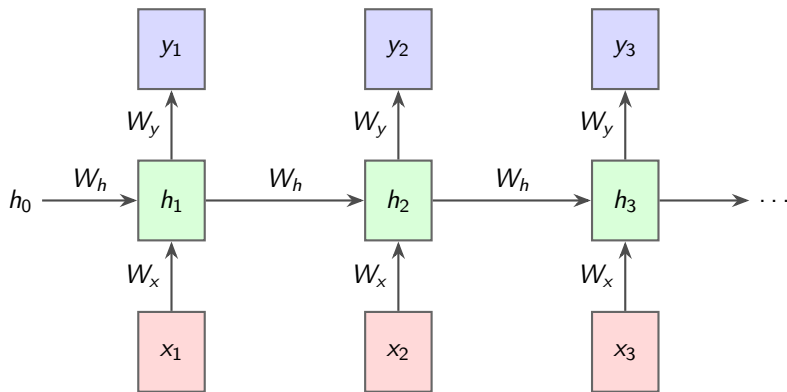
Time Step $t = 1$: Input $x_1 = 2.0$

$$\begin{aligned}h_1 &= \tanh(W_h h_0 + W_x x_1 + b_h) \\&= \tanh((-1.0)(0.0) + (0.5)(2.0) + 0.1) \\&= \tanh(0 + 1.0 + 0.1) = \tanh(1.1) \approx \mathbf{0.80}\end{aligned}$$

Time Step $t = 2$: Input $x_2 = 1.0$

$$\begin{aligned}h_2 &= \tanh(W_h h_1 + W_x x_2 + b_h) \\&= \tanh((-1.0)(0.80) + (0.5)(1.0) + 0.1) \\&= \tanh(-0.80 + 0.50 + 0.1) = \tanh(-0.20) \approx \mathbf{-0.20}\end{aligned}$$

Visualizing the Full Many-to-Many Architecture



Generating Outputs from Hidden States

To make predictions (many-to-many architecture), we compute an output $\mathbf{y}_t$ at each step based on the hidden state.

Output Equation

$$\mathbf{y}_t = \sigma_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

- $\mathbf{W}_y, \mathbf{b}_y$: Output weight matrix and bias (also shared across time).
- σ_y : Output activation (e.g., Sigmoid for binary classification, Softmax for next word prediction).

Continuing our numeric example: Let $W_y = 2.0$, $b_y = -0.5$, $\sigma_y = \text{Sigmoid}$.

$$y_1 = \text{Sigmoid}(2.0(\textcolor{green}{0.80}) - 0.5) = \text{Sigmoid}(1.1) \approx \textcolor{blue}{0.75}$$

$$y_2 = \text{Sigmoid}(2.0(\textcolor{green}{-0.20}) - 0.5) = \text{Sigmoid}(-0.9) \approx \textcolor{blue}{0.29}$$

Numeric Example: 2D Many-to-Many Architecture

2D Parameters

$$\mathbf{W}_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

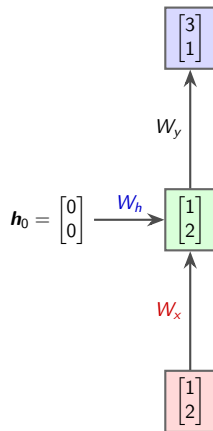
$$\mathbf{W}_h = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\mathbf{W}_y = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Rules ($b = 0$):

$$\mathbf{h}_t = \text{ReLU}(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$$

$$\mathbf{y}_t = \mathbf{W}_y \mathbf{h}_t$$



Numeric Example: 2D Many-to-Many Architecture

2D Parameters

$$\mathbf{W}_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

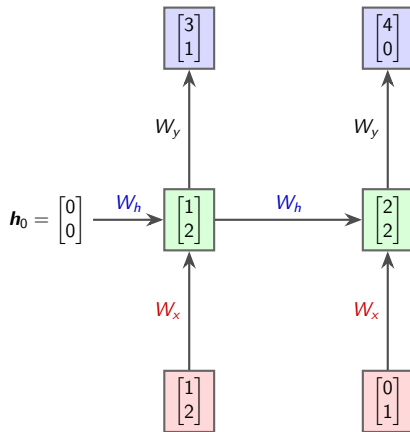
$$\mathbf{W}_h = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\mathbf{W}_y = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Rules ($b = 0$):

$$\mathbf{h}_t = \text{ReLU}(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$$

$$\mathbf{y}_t = \mathbf{W}_y \mathbf{h}_t$$



Numeric Example: 2D Many-to-Many Architecture

2D Parameters

$$\mathbf{W}_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

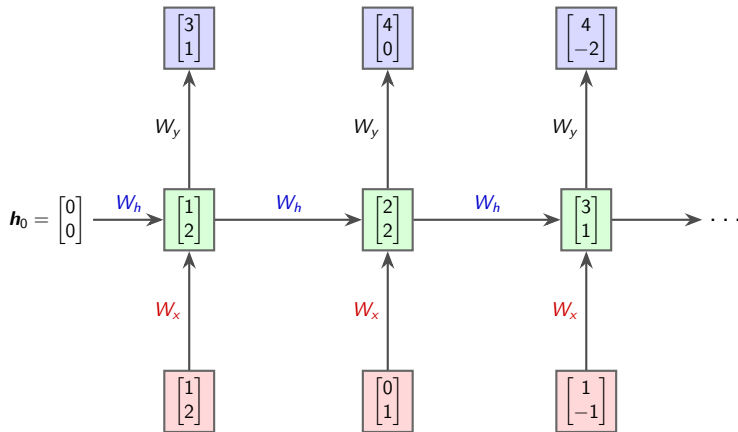
$$\mathbf{W}_h = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\mathbf{W}_y = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Rules ($b = 0$):

$$\mathbf{h}_t = \text{ReLU}(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$$

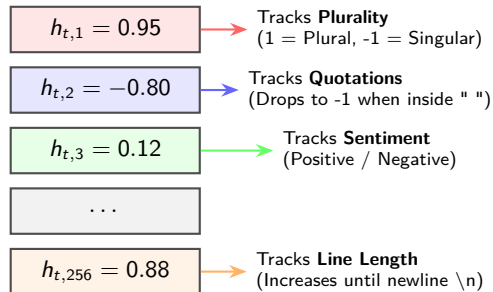
$$\mathbf{y}_t = \mathbf{W}_y \mathbf{h}_t$$



Intuition: Inside the Hidden State Vector

Each position (neuron) in the vector $\mathbf{h}$ acts as a memory cell tracking a specific feature of the sequence.

Imagine a hidden state vector $\mathbf{h}_t \in \mathbb{R}^{256}$. Each of the 256 dimensions learns to track something useful for minimizing the loss:



Note: While some neurons are highly interpretable, most information is "distributed" (entangled) across multiple dimensions.

Character-level Language Model Sampling

Vocabulary:

idx 0: h, idx 1: e

idx 2: l, idx 3: o

RNN Parameters

$$W_{xh} = \begin{bmatrix} 0.5 & -0.1 & 0.2 & 0.0 \\ 0.2 & 0.4 & -0.1 & 0.5 \\ 0.1 & -0.2 & 0.3 & 0.1 \end{bmatrix}$$

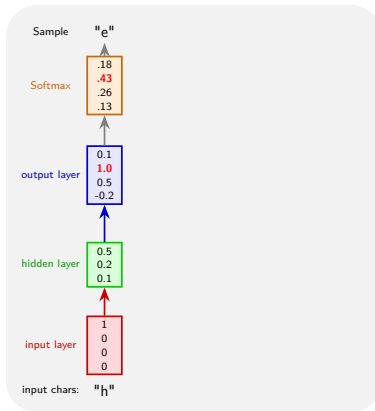
$$W_{hh} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & -5 \\ 1 & 2 & 1 \\ 0 & -1 & 3 \end{bmatrix}$$

$$b_y = \begin{bmatrix} 0.1 \\ 0.0 \\ -0.5 \\ -0.3 \end{bmatrix}, h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$h_t = \text{ReLU}(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t + b_y$$



At test-time sample characters one at a time, feed back to model.

Character-level Language Model Sampling

Vocabulary:

idx 0: h, idx 1: e

idx 2: l, idx 3: o

RNN Parameters

$$W_{xh} = \begin{bmatrix} 0.5 & -0.1 & 0.2 & 0.0 \\ 0.2 & 0.4 & -0.1 & 0.5 \\ 0.1 & -0.2 & 0.3 & 0.1 \end{bmatrix}$$

$$W_{hh} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

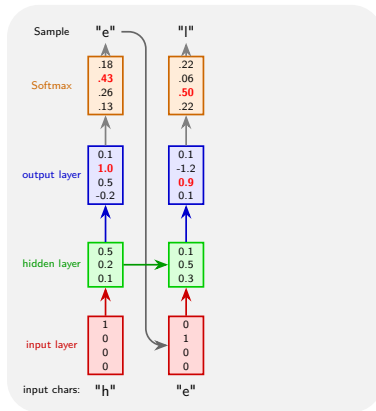
$$W_{hy} = \begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & -5 \\ 1 & 2 & 1 \\ 0 & -1 & 3 \end{bmatrix}$$

$$b_y = \begin{bmatrix} 0.1 \\ 0.0 \\ -0.5 \\ -0.3 \end{bmatrix}, h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$h_t = \text{ReLU}(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t + b_y$$

At test-time sample characters one at a time,
feed back to model.



Character-level Language Model Sampling

Vocabulary:

idx 0: h, idx 1: e

idx 2: l, idx 3: o

RNN Parameters

$$W_{xh} = \begin{bmatrix} 0.5 & -0.1 & 0.2 & 0.0 \\ 0.2 & 0.4 & -0.1 & 0.5 \\ 0.1 & -0.2 & 0.3 & 0.1 \end{bmatrix}$$

$$W_{hh} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

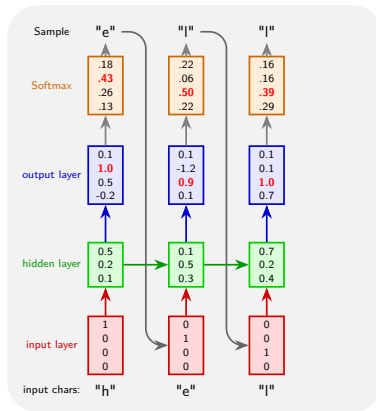
$$W_{hy} = \begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & -5 \\ 1 & 2 & 1 \\ 0 & -1 & 3 \end{bmatrix}$$

$$b_y = \begin{bmatrix} 0.1 \\ 0.0 \\ -0.5 \\ -0.3 \end{bmatrix}, h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$h_t = \text{ReLU}(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t + b_y$$

At test-time sample characters one at a time,
feed back to model.



Character-level Language Model Sampling

Vocabulary:

idx 0: h, idx 1: e

idx 2: l, idx 3: o

RNN Parameters

$$W_{xh} = \begin{bmatrix} 0.5 & -0.1 & 0.2 & 0.0 \\ 0.2 & 0.4 & -0.1 & 0.5 \\ 0.1 & -0.2 & 0.3 & 0.1 \end{bmatrix}$$

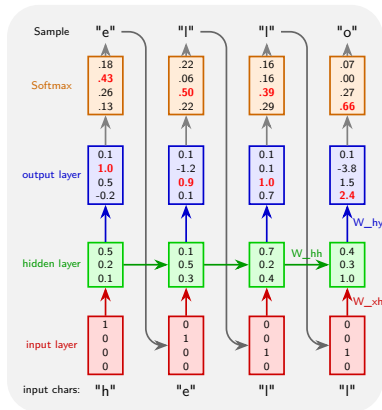
$$W_{hh} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & -5 \\ 1 & 2 & 1 \\ 0 & -1 & 3 \end{bmatrix}$$

$$b_y = \begin{bmatrix} 0.1 \\ 0.0 \\ -0.5 \\ -0.3 \end{bmatrix}, h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$h_t = \text{ReLU}(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t + b_y$$



At test-time sample characters one at a time,
feed back to model.

Training: The Loss Function

For a **Many-to-Many** sequence of length T :

- At every time step t , we compute a prediction $\hat{y}_t$.
- We compare it to the true target y_t to compute a local loss L_t .
- **Sequence Loss:** Sum (or average) of losses across all time steps.

$$L_{\text{seq}} = \sum_{t=1}^T L_t(\hat{y}_t, y_t)$$

- **Batch Loss:** The sequence is split into smaller time windows. The loss is calculated only over the active chunk:

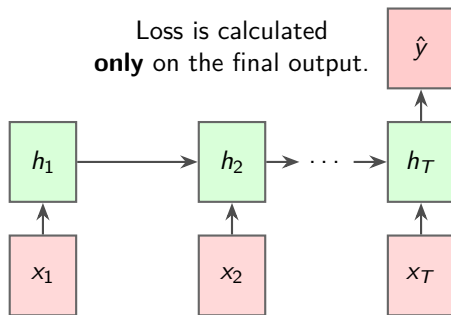
$$\mathcal{L}_{\text{batch}} = \sum_{t=t_{\text{start}}}^{t_{\text{end}}} L_t(\hat{y}_t, y_t)$$

Many-to-One Architecture

Not all tasks require an output at every time step.

Examples:

- **Sentiment Analysis:** "This movie is terrible" $\rightarrow$ [RNN] $\rightarrow$ **Negative**.
- **Translation Encoder:** Process Spanish sentence $\rightarrow$ [RNN] $\rightarrow$ Final context vector.



The Problem of Long-Term Dependencies

RNNs struggle to propagate the gradient signal over long sequences due to the **Vanishing/Exploding** gradients problem.

Recall: $\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h)$

By the chain rule:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} = \prod_{i=k+1}^T \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_{i=k+1}^T (\mathbf{W}_h^T \text{diag}(\sigma'(\dots)))$$

- If the largest singular value of $\mathbf{W}_h > 1$: gradients **explode**.
- If the largest singular value of $\mathbf{W}_h < 1$: gradients **vanish**.

Because $\mathbf{W}$ is shared across all time steps, the total gradient of the loss with respect to $\mathbf{W}$ is the sum of contributions from each time step:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{k=1}^T \frac{\partial L}{\partial \mathbf{h}_T} \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}}$$

Long Short-Term Memory (LSTM) Networks

To solve this, LSTMs introduce a **cell state** ($\mathbf{c}_t$) that acts like a "conveyor belt", allowing gradients to flow almost unchanged. The flow of information is regulated by learnable **gates**.

1. **Forget Gate:** What to erase from memory.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

2. **Input Gate:** What new info to store.

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

3. **Candidate Cell State:** New values.

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

Cell State Update

Combine old memory with new candidate:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

4. **Output Gate:** What parts of memory to output.

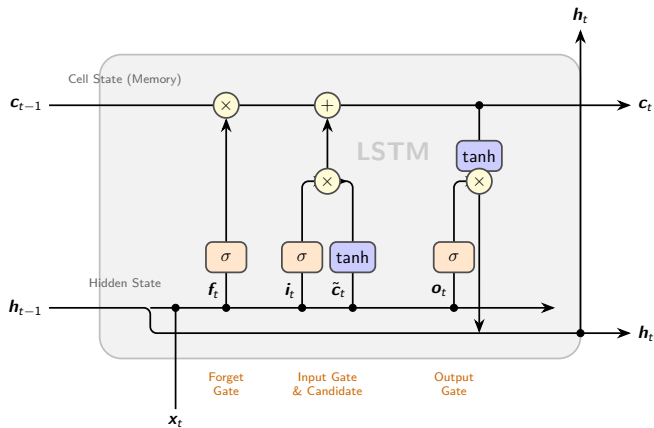
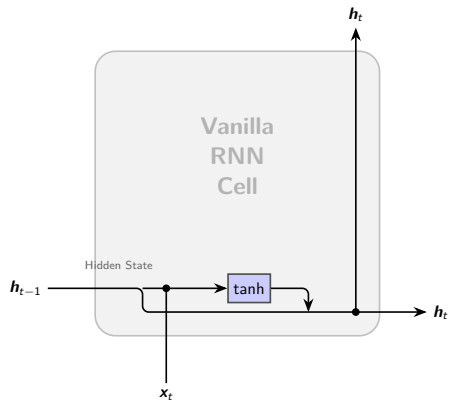
$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

5. **Hidden State:** Filtered memory.

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Note: $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ denotes concatenation. $\odot$ is the Hadamard (element-wise) product. σ squashes values to (0, 1).

Anatomy: Vanilla RNN vs. LSTM Cell



Intuition for LSTM)

In a Vanilla RNN, the hidden state $\mathbf{h}_t$ is **overloaded**. It has to do two jobs at once:

- 1 Store the entire long-term history of the sequence.
- 2 Provide the exact features needed to make a prediction right now.

LSTMs solve this by splitting these jobs into a **Private Memory** and a **Public Output**:

1. Cell State ($\mathbf{c}_t$): Private Memory

- **The Diary:** Contains *everything* the network has seen and deemed worth keeping.
- **Gradient Highway:** Updates are mostly linear (addition). This prevents gradients from vanishing during BPTT.

2. Hidden State ($\mathbf{h}_t$): Public Output

- **The Spoken Word:** A filtered subset of $\mathbf{c}_t$.
- **Immediate Focus:** The Output Gate ($\mathbf{o}_t$) reveals only the memories relevant for the *current* step's prediction.

Note that $\tilde{\mathbf{c}}_t$ is the exact equivalent to $\mathbf{h}_t$ in RNN. But instead of using this directly as the output (as in RNN), the LSTM uses it as a suggestion of what to add to the private memory ($\mathbf{c}_t$).

Numeric Example: 1D LSTM Forward Pass

- **Inputs:** $x_1 = 1.0$ **Previous States:** $h_0 = 0.0$, $c_0 = 0.5$
- **Weights:** $W_{fx} = 1.0$, $W_{ix} = 0.0$, $W_{cx} = 1.0$, $W_{ox} = 1.0$ (All biases = 0)

Step 1: Compute Gates

Forget Gate:

$$f_1 = \sigma(1.0 \cdot x_1) = \sigma(1.0) \approx \mathbf{0.73}$$

Input Gate:

$$i_1 = \sigma(0.0 \cdot x_1) = \sigma(0.0) = \mathbf{0.50}$$

Candidate:

$$\tilde{c}_1 = \tanh(1.0 \cdot x_1) = \tanh(1.0) \approx \mathbf{0.76}$$

Output Gate:

$$o_1 = \sigma(1.0 \cdot x_1) = \sigma(1.0) \approx \mathbf{0.73}$$

Step 2: Update Cell State

$$c_1 = (f_1 \cdot c_0) + (i_1 \cdot \tilde{c}_1)$$

$$c_1 = (0.73 \cdot 0.50) + (0.50 \cdot 0.76)$$

$$c_1 = 0.365 + 0.380$$

$$c_1 = \mathbf{0.745}$$

(Memory belt slightly increased)

Step 3: Update Hidden State

$$h_1 = o_1 \cdot \tanh(c_1)$$

$$h_1 = 0.73 \cdot \tanh(0.745)$$

$$h_1 = 0.73 \cdot 0.63$$

$$h_1 \approx \mathbf{0.46}$$

Numeric Example: 2D LSTM Forward Pass

Inputs & Initial States:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{c}_0 = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}$$

Weight Matrices ($\mathbf{W}_x$):

Note: $\mathbf{W}_h$ terms evaluate to 0 since $\mathbf{h}_0 = \mathbf{0}$.

Assume bias $\mathbf{b} = \mathbf{0}$.

$$\mathbf{W}_f = \begin{bmatrix} 1.4 & 0 \\ -1.4 & 0 \end{bmatrix}$$

$$\mathbf{W}_i = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{W}_c = \begin{bmatrix} 3.0 & 0 \\ -3.0 & 0 \end{bmatrix}$$

$$\mathbf{W}_o = \begin{bmatrix} 5.0 & 0 \\ 0 & 0 \end{bmatrix}$$

Step 1: Compute Gates (σ or $\tanh$ of $\mathbf{W}\mathbf{x}_1$)

$$\mathbf{f}_1 \approx \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \quad \mathbf{i}_1 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

$$\tilde{\mathbf{c}}_1 \approx \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix}, \quad \mathbf{o}_1 \approx \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

Step 2: Update Cell State

$$\mathbf{c}_1 = \mathbf{f}_1 \odot \mathbf{c}_0 + \mathbf{i}_1 \odot \tilde{\mathbf{c}}_1$$

$$\mathbf{c}_1 = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix} \odot \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \odot \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix}$$

$$\mathbf{c}_1 = \begin{bmatrix} 0.4 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} \mathbf{0.9} \\ \mathbf{-0.3} \end{bmatrix}$$

Step 3: Update Hidden State

Assume $\tanh(0.9) \approx 0.7$ and $\tanh(-0.3) \approx -0.3$

$$\mathbf{h}_1 = \mathbf{o}_1 \odot \tanh(\mathbf{c}_1)$$

$$\mathbf{h}_1 \approx \begin{bmatrix} 1.0 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 0.7 \\ -0.3 \end{bmatrix} = \begin{bmatrix} \mathbf{0.70} \\ \mathbf{0} \end{bmatrix}$$

The "AlexNet" of RNNs: Google's Deep Seq2Seq (2014)

Architecture Specifications

Task: English $\rightarrow$ French Translation

1. Vocabulary Details:

- Source (English): 160,000 words
- Target (French): 80,000 words

2. Dimensionality:

- **Word Embeddings:** 1,000 dimensions
- **Hidden States (h & c):** 1,000 units

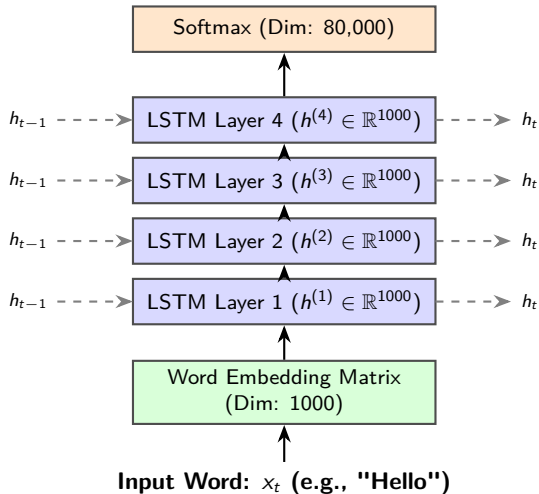
3. Network Depth:

- **Encoder:** 4 stacked LSTM layers
- **Decoder:** 4 stacked LSTM layers

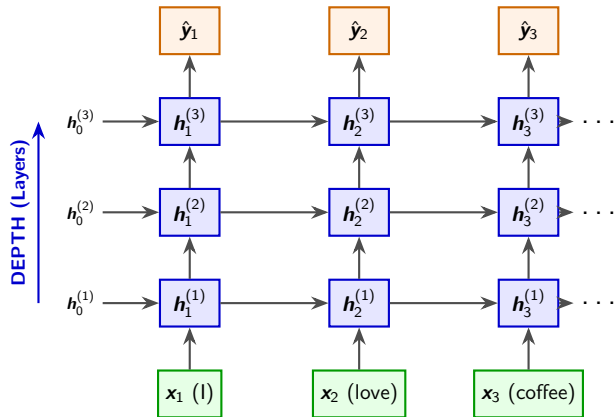
4. Total Scale:

- **Parameters:** $\sim$ 380 Million
- **Training:** 10 days on 8-GPU cluster

Vertical Slice at Time t (Decoder)



Stacked LSTM



Bi-LSTM

